

Software Architecture
(T630019402)

Lecture #7, April 7, 2026

Jukka Ruohonen

Reflections

- ▶ **Attendance was less than 10%** during the previous lecture
- ▶ It seems only a **single person has done exercises** twice or so
- ▶ Recall that the **exam will be 3,5 – 4,0 hours** at maximum, and *failing at the exam will delay your graduation*
 - Recall also that one ECTS is 27h of full-time work
- ▶ Given the last point, we will have simple **mock-up questions** today and in the subsequent lectures; these resemble something that may appear in the exam but not perfectly because we can only go through them orally

Informal Feedback

1. What **has worked** thus far?
 2. What **has not worked** thus far?
 3. What would you **change**?
-
- ▶ Yet, recall the nature of the course

Later Evaluation (1/2)

- ▶ With attendance having been so poor, the SDU's formal course evaluation at the end of the course will presumably be *biased and invalid*; hence, to improve things, I would encourage **only those who have attended to answer**
- ▶ Before the last lecture on 5 May, there will be a **custom survey** involving three questions on:
 1. What **grade** are you expecting?
 2. How many **lectures** did you attend?
 3. Did you do a single **exercise**?
- ▶ Of these, correlating (1) and the actual grades gives data about your ability to evaluate your learning on your own, whereas (2) and the actual grades are known to correlate heavily and robustly according to numerous scientific articles

Later Evaluation (2/2)

- ▶ The fourth question in the survey will be about my plans for pedagogical interventions for the next year
- ▶ To this end, I would love to hear your opinions about my plan to change the course so that about **30–40% of the grade** will come from **weekly multiple choice questions** on previously covered topics
- ▶ The questions will resemble something like the mockup questions covered from today onward

Case #6 Presentations

- ▶ Skipped due to the reasons mentioned

A Few Short Mock-up Questions (1/9)

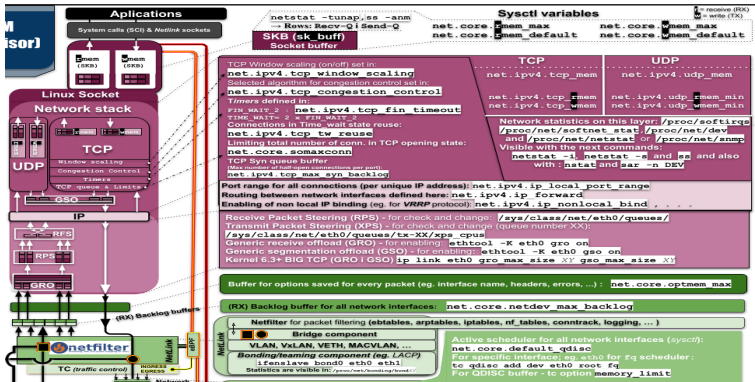


Figure 1: Please Name At Most Two Fundamental Things This Figure Might Represent from a Software Architecture Perspective?

A Few Short Mock-up Questions (2/9)

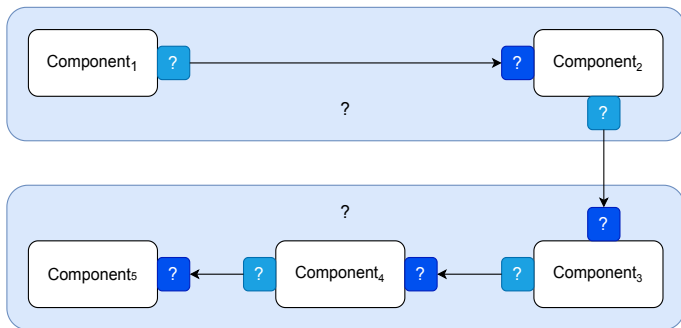


Figure 2: (a) Name Two Architectural Patterns Used in the Figure, and
(b) Explain What the Smaller Rectangles Colored in Darker Blue Mean?

A Few Short Mock-up Questions (3/9)

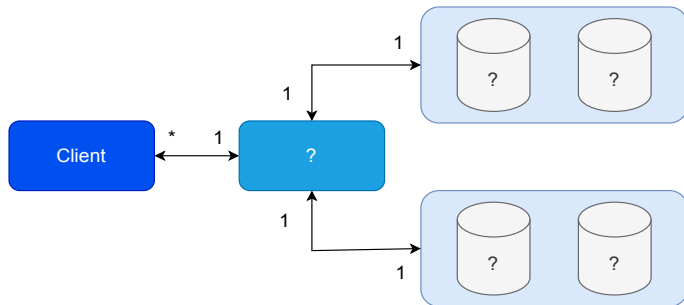


Figure 3: (a) What Might Be the Rationale for This Design, and
(b) What the Question Marks Could Denote for Making It Explicit?

A Few Short Mock-up Questions (4/9)

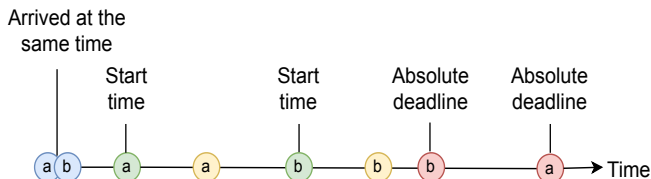


Figure 4: (a) What a Scheduler Behind This Diagram Might Be Called, and (b) Why It Is Generally a Bad Idea for Real-Time Systems?

A Few Short Mock-up Questions (5/9)

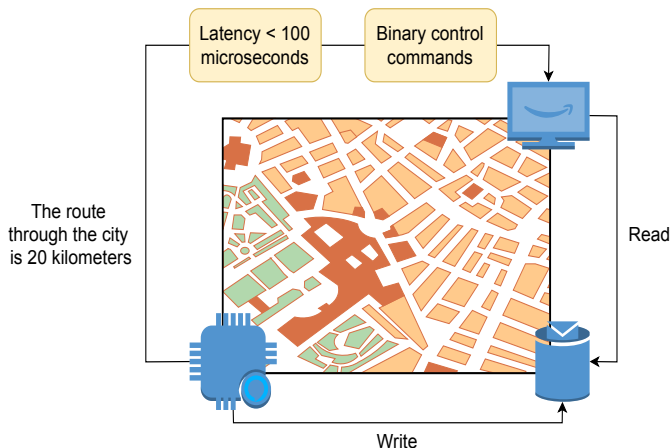


Figure 5: (a) What Distance Concepts Are Involved in the Diagram, (b) What Architectural Pattern Is Used in the Design and (c) Why It Might Be Used, and (d) What Quality Attributes Might Be Involved?

A Few Short Mock-up Questions (6/9)

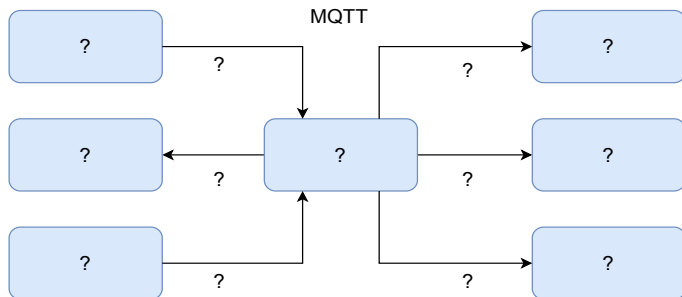


Figure 6: (a) What Architectural Pattern Does the Diagram Represent and (b) What the Question Marks Could Denote for Making It Explicit

A Few Short Mock-up Questions (7/9)

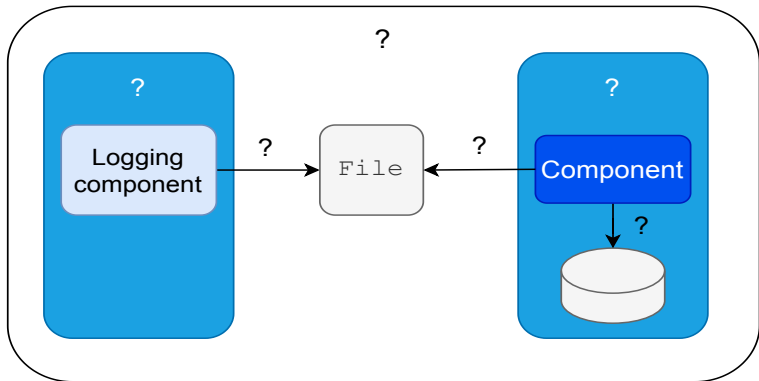


Figure 7: (a) What the Question Marks Could Denote and (b) Why?

A Few Short Mock-up Questions (8/9)

- ▶ What is the difference between performance and scalability?

A Few Short Mock-up Questions (9/9)

- ▶ Thus, at least if you have attended, nothing hard as such?
 - If you have not attended, do your homework
- ▶ Obviously: if I ask about (a), (b), and (c), say, and you can only answer to (b), it will be 1/3 points for the given question
- ▶ Please also pay attention to what I write; if I ask to justify a decision with **at most two sentences** and you answer with five, you will get zero points from the given question
 - If I further emphasize **prioritization**, for instance, one of the sentences should focus on that; you will get zero points if your answer has no rationale whatsoever when a rationale is required (as it is for a prioritization)

Today's Lecture

- ▶ Today's lecture is about **scalability** as a distinct QA
- ▶ First recall that scalability as a term is used also in many other contexts, including in economics and business
- ▶ Then recall that a system is scalable if performance can be **proportionally** increased by increasing resources
 - Therefore, it is a good idea to start from what proportionality can be seen to mean

Scaling and Proportionality (1/3)

- ▶ Amdahl's (1967) "law"¹ can be expressed as: $y = \frac{1}{a + \frac{1-a}{p}}$
- ▶ In the equation y is the speedup, p is the number of CPUs in a machine or a larger distributed system of machines, $1 - a$ is a share of computations that can run in parallel, and a is a share that cannot be parallelized

¹ Interestingly enough, Amdahl's original (1967) paper does not have any equations or formalism at all; the equation shown is by Gustafson (1988).

Scaling and Proportionality (2/3)

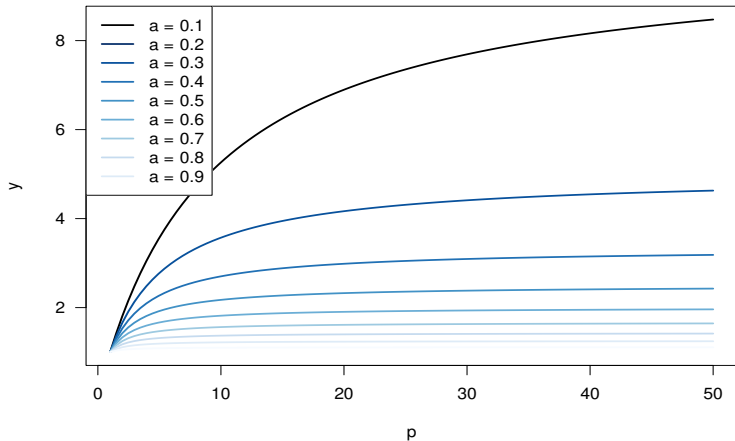


Figure 8: Amhdal's Law in Action

Scaling and Proportionality (3/3)

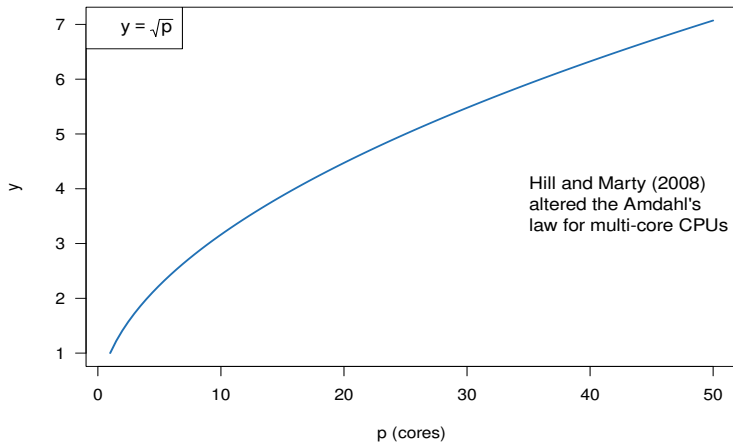


Figure 9: The Proposal of Hill & Marty (2008)

Scalability Tactics: Service Meshes

- ▶ A **service mesh** is a high-level scalability tactic
 - Also, the terminology is vague, often varying from a vendor to another; hence, without a loss of specificity, we can equate service meshes generally to **cluster computing** that is a domain of **high-performance computing** (HPC)
- ▶ In what follows, **Kubernetes** is used as a reference point, although many alternatives exist too (usually with their own equally vague terminology)

Kubernetes: Parts and Terms (1/3)

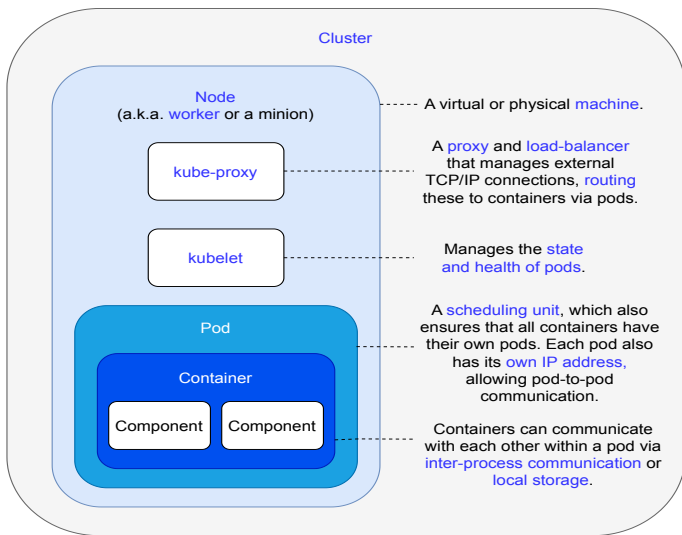


Figure 10: Kubernetes' Core Parts and Terms

Kubernetes: Parts and Terms (2/3)

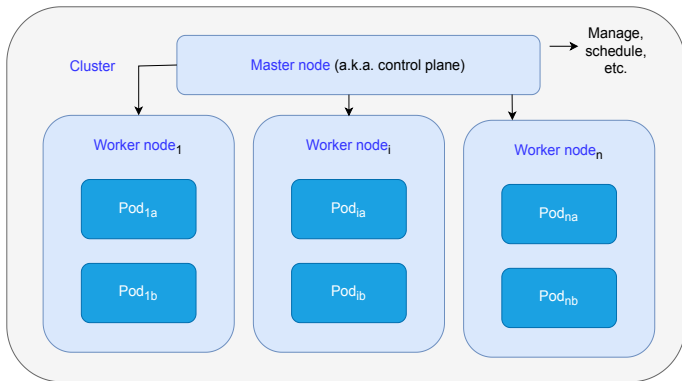


Figure 11: Kubernetes' Further Core Parts and Terms

Kubernetes: Parts and Terms (3/3)

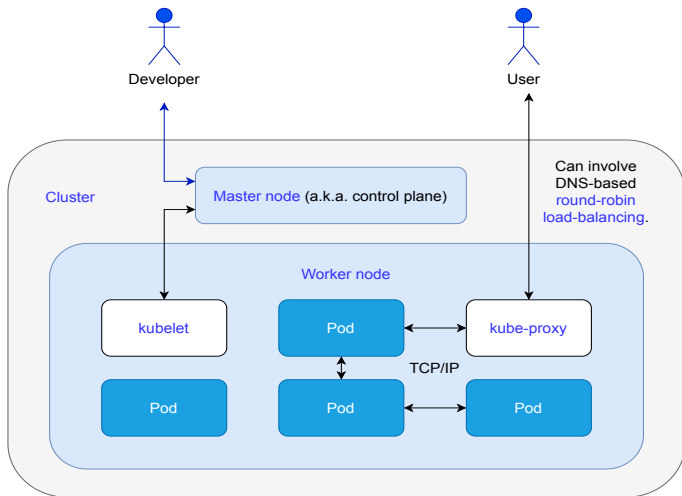


Figure 12: And Still Further Kubernetes' Core Parts and Terms

Kubernetes: Communication (1/4)

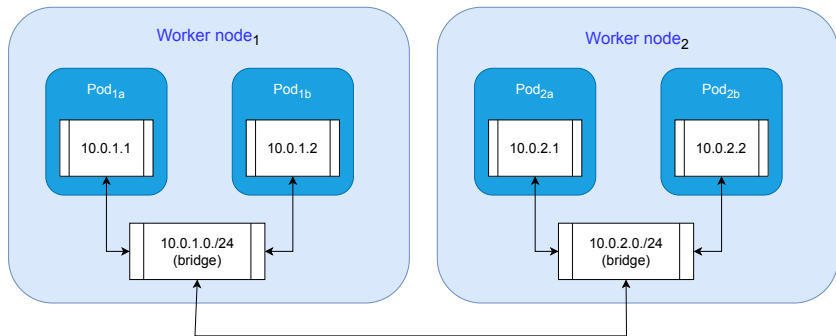


Figure 13: An Example of Kubernetes' Pod-to-Pod Communication

Kubernetes: Communication (2/4)

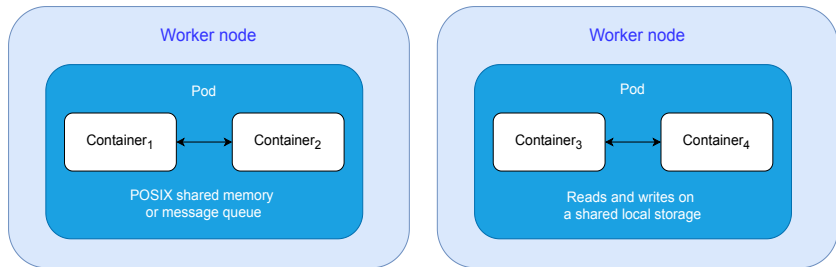


Figure 14: Container-to-Container Communication in Kubernetes

Kubernetes: Communication (3/4)

- ▶ From the previous two examples we can deduce that *frequently interacting components should be placed into their containers within a same pod* already because inter-process communication (IPC) is faster than TCP/IP communication
 - The point generalizes into a **gateway pattern**
- ▶ In addition, a so-called **sidecar pattern** groups all cross-cutting concerns into their own pods or containers
 - The rationale of such **co-locating** is clear also because cross-cutting concerns often lead to complexity and poor maintainability due to duplication of code

Kubernetes: Communication (4/4)

- ▶ A gateway can also be used to handle communication between components (services) that are otherwise incompatible

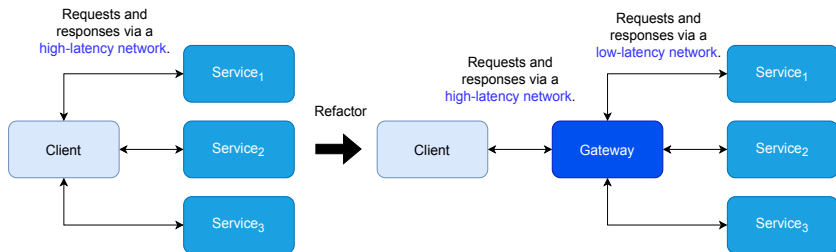


Figure 15: The Gateway Pattern (adopted from Microsoft 2025)

Kubernetes: Sidecar (1/4)

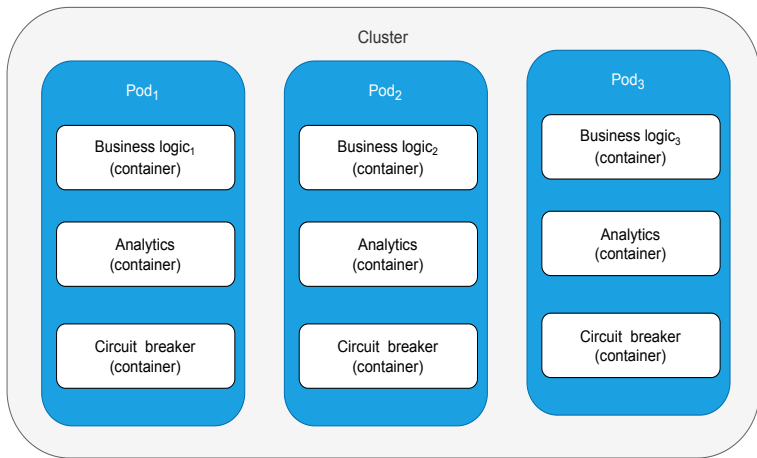


Figure 16: No Sidecar (adopted from Maia & Correia 2022)

Kubernetes: Sidecar (2/4)

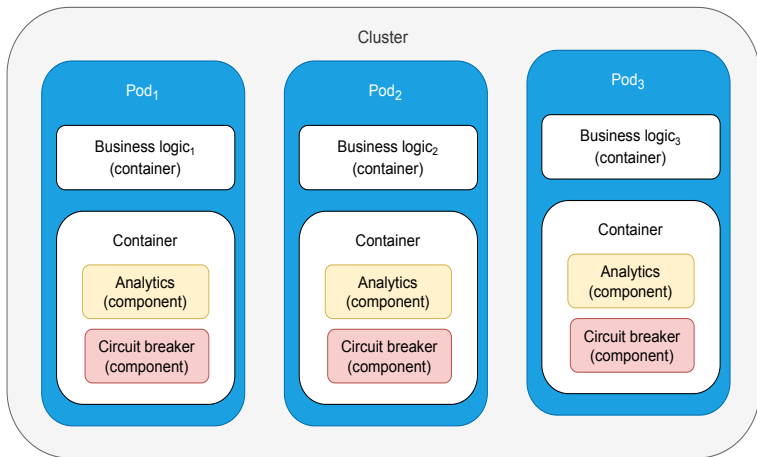


Figure 17: A Sidecar (adopted from Maia & Correia 2022)

Kubernetes: Sidecar (3/4)

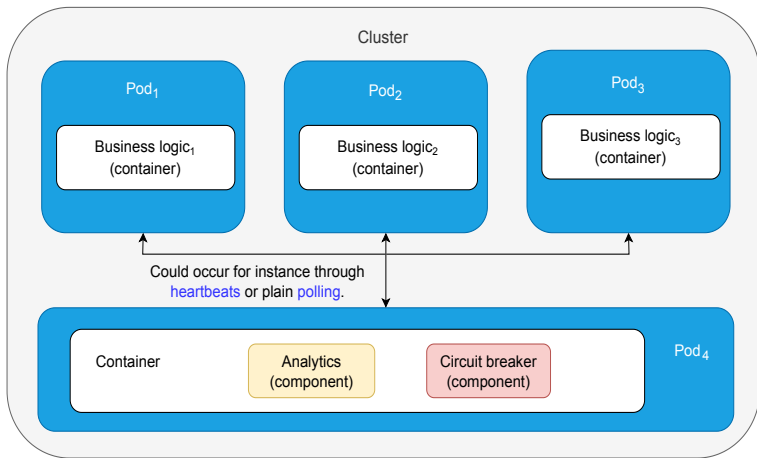


Figure 18: Another Sidecar (adopted from Microsoft 2020)

Kubernetes: Sidecar (4/4)

- ▶ It can be concluded from the previous three examples that the design space is wide but complexity may quickly escalate
- ▶ In addition – or perhaps even as an alternative, Google (2024) recommends keeping the **sizes of nodes and containers small**, while this abstracting enough to meet other, non-scalability QAs
 - The reason is that if deployed in a hyperscaler cloud, various scheduling options and so-called **autoscalers** need to download their images when deploying new replicated nodes and containers for meeting load
 - Thus, it is not clear whether the analytics and circuit breaker components in Fig. 18 should be within their own containers or even pods

Kubernetes: Scaling (1/7)

- ▶ There are two analytical ways to frame and understand scaling of Kubernetes clusters:
 1. **Horizontal scaling** means that new machines or new pods (replicas) are added to meet increased load
 2. **Vertical scaling** means that more resources (memory, CPUs, etc.) are added to existing machines to meet increased load
- ▶ It is difficult to interpret what these might mean in terms of the Amdahl's law, but, nevertheless, *diminishing returns* seem a reasonable assumption *at some point* (cf. also costs)
- ▶ By default, Kubernetes (2024a) comes with an autoscaler that also scales down horizontal or vertical upward scaling once load has decreased

Kubernetes: Scaling (2/7)

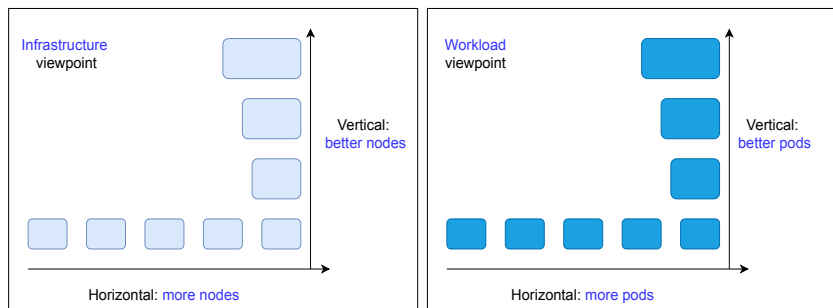


Figure 19: Scaling of (Kubernetes) Clusters (adopted from Google 2024)

Kubernetes: Scaling (3/7)

- ▶ Most of the autoscalers use different metrics (such as CPU usage) for determining appropriate upward scaling
- ▶ They also provide **minimums** (e.g., 10 machines with 100 pods must always be running) and **maximums** (e.g., no more than 1000 machines may be deployed)
 - It is a good idea to define these to avoid cost surprises
- ▶ Given that the autoscalers are dynamic algorithms, Google (2024) recommends also to **minimize startup and shutdown times** because both add overhead
- ▶ While **logging** is necessary (and also for monitoring a cluster), also these add to costs when done excessively in huge clusters

Kubernetes: Scaling (4/7)

- ▶ Besides the maximums and minimums, it is a good idea to return to the Amdahl's law and what was said in the previous lecture about performance
- ▶ Indeed, Dinesh (2018) warns about a temptation to try to fix latency issues by adding more resources to a Kubernetes cluster instead of looking at whether the **code is bad**
 - In addition, he hints that **configuration problems** are common, which partially reflects the **complexity of Kubernetes** and cluster computing in general
 - Configuration bugs cause also security issues (Bufalino 2025)

Kubernetes: Scaling (5/7)

- ▶ But **how large is a large cluster?**
- ▶ Kubernetes' (2024b) documentation notes the following upper limits for supported clusters:
 1. No more than 110 pods per node
 2. No more than 5,000 nodes
 3. No more than 150,000 total pods
 4. No more than 300,000 total containers
- ▶ So large is quite large indeed, yet not extremely large
 - For instance, the top-10 world LUMI supercomputer in Finland has currently **362,496 CPU cores**, but computing with it likely occurs in smaller clusters

Kubernetes: Scaling (6/7)

- ▶ In addition, hyperscaling clouds can use **geofencing** to route clients to closer geographic locations, including data centers
 - There is no point to serve Danish clients from Morocco
- ▶ That said, geofencing is a more general class of algorithms and architectural tactics applicable to smaller cases, including, say, smart cities or specific IoT deployments
- ▶ In addition, geography can matter for availability and fault-tolerance of systems; Ukraine is a good example
- ▶ However, note that cluster and cloud computing in general usually opt only for **eventual consistency**; not every read returns (or needs to return) the most recent write

Kubernetes: Scaling (7/7)

- ▶ *When do you need to really consider what was said?*
- ▶ The obvious answer is that it depends on a context; scientific computing may really need a HPC cluster, continuous streams from sensors may need a cloud as may a service for users to upload high-quality videos, and so forth and so on
- ▶ However, hyperscalers are costly; for smaller businesses and traditional “Internet applications”, you should be able to serve something like +100,000 (or more!) users without a cloud
- ▶ Recall also what was said in the second lecture; scaling a business first is arguably more important than scaling a system

Kubernetes: Monitoring (1/2)

- ▶ Prometheus can be recommended for monitoring Kubernetes and many other large systems, including commercial clouds

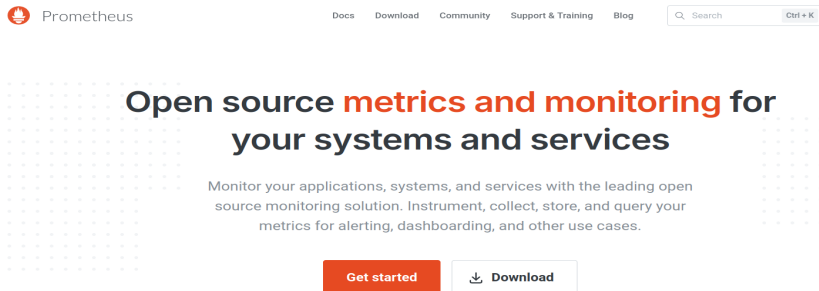


Figure 20: <https://prometheus.io/>

Kubernetes: Monitoring (2/2)

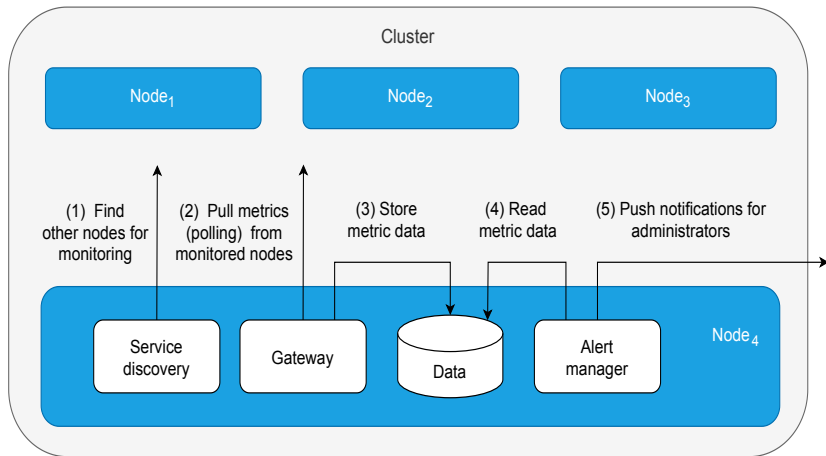


Figure 21: An Incomplete Sketch for Monitoring with Prometheus

Load Balancing Revisited: DNS (1/6)

- ▶ The previous examples about scaling Kubernetes clusters are essentially about **dynamic algorithms** that are common also in more general-purpose load balancing
 - The *throttling* and *scheduling* algorithms from the previous lecture are also good examples of dynamic algorithms
- ▶ In contrast, **static algorithms** distribute load across systems without taking into account their current states or health
- ▶ The Domain Name System (DNS) is a classical example still partially but fundamentally relying on static algorithms; many (but not all!) Internet servers distribute via name servers addresses to clients in rotation using **round-robin** scheduling that were introduced in the previous lecture

Load Balancing Revisited: DNS (2/6)

```
jruoho@adorno~$ nslookup -q=A netflix.com && sleep 2 && nslookup -q=A netflix.com
Server:      127.0.0.1
Address:     127.0.0.1#53
```

Non-authoritative answer:

```
Name:   netflix.com
Address: 3.251.50.149
Name:   netflix.com
Address: 54.155.178.5
Name:   netflix.com
Address: 54.74.73.31
```

```
Server:      127.0.0.1
Address:     127.0.0.1#53
```

Non-authoritative answer:

```
Name:   netflix.com
Address: 54.74.73.31
Name:   netflix.com
Address: 3.251.50.149
Name:   netflix.com
Address: 54.155.178.5
```

Figure 22: Two A Record (IPv4 Address) Queries for Netflix

Load Balancing Revisited: DNS (3/6)

- ▶ While simple, DNS-based round-robin scheduling provides scalability because clients are routed to different servers
 - Although such round-robin scheduling continues to route requests also to dead or otherwise non-functioning servers (because there are no heartbeats or other monitoring), it does still provide some coarse **fail-over** for clients because they will pick a subsequent address (server) after a timeout
 - In contrast, the *active-active* and *active-passive* (“standby”) load-balancing scenarios from the fifth lecture require heartbeats, polling, or other monitoring
- ▶ Some DNS functionality, such as for email, use more complex but still static (‘or “semi-static”) algorithms

Load Balancing Revisited: DNS (4/6)

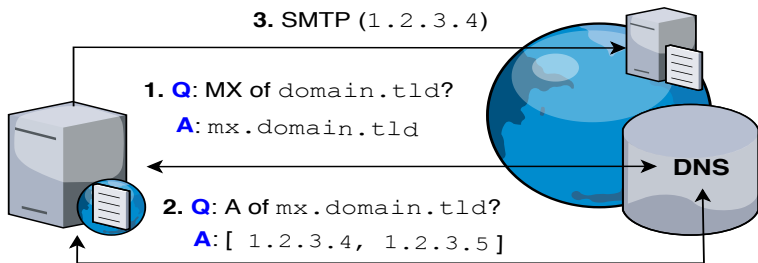


Figure 23: Adopted from Ruohonen (2020, Fig. 1)

Load Balancing Revisited: DNS (5/6)

Although we cannot deduce the actual deployment configuration from the previous figure, the **case B** would be the logical one in terms of scalability (and fault-tolerance too).

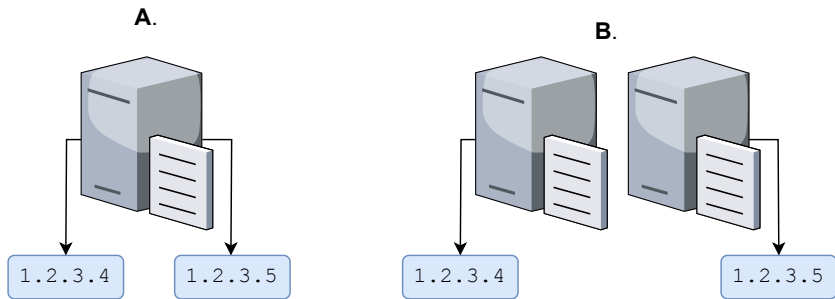


Figure 24: Two Example Configurations for the Previous Figure

Load Balancing Revisited: DNS (6/6)

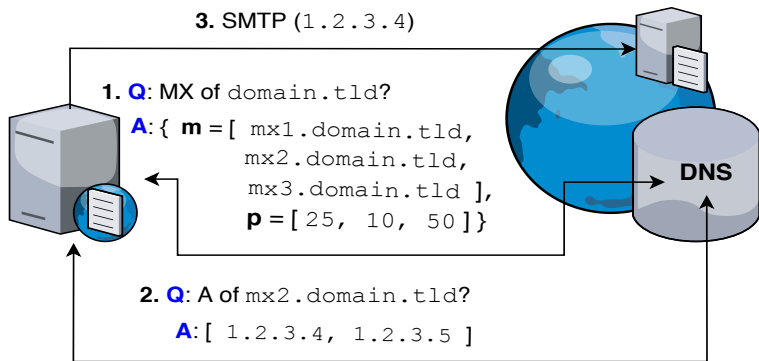


Figure 25: Adopted from Ruohonen (2020, Fig. 2)

Load Balancing Revisited: CDNs (1/5)

- ▶ **Content delivery networks** (CDNs) are further popular cloud-like services for scaling web applications in particular
 - Akamai and Cloudflare are currently the biggest “traditional CDN companies” in the West, but there are many smaller companies too, and some large companies and telecommunications operators have their own CDNs
- ▶ In addition, many large-scale CDNs, such as Cloudflare, offer also denial of service (DoS) protection, web application firewalls (WAFs), and much more

Load Balancing Revisited: CDNs (2/5)

- ▶ CDNs are configured via DNS: a host aliases its DNS records (typically A for IPv4 and AAAA for IPv6) for those of a CDN operator; hence, clients are routed to the CDN's servers who cache and thus host content on behalf of the origin host

```
jruoho@adorno~$ host portal.findresearcher.sdu.dk
portal.findresearcher.sdu.dk is an alias for findresearchersdu.elsevierpure.com.
findresearchersdu.elsevierpure.com is an alias for eu.prod.elsevierpure.com.
eu.prod.elsevierpure.com is an alias for eu.prod.elsevierpure.com.cdn.cloudflare.net.
eu.prod.elsevierpure.com.cdn.cloudflare.net has address 172.64.148.16
eu.prod.elsevierpure.com.cdn.cloudflare.net has address 104.18.39.240
```

Figure 26: Parts of SDU, Elsevier, and Cloudflare Aliased Together

Load Balancing Revisited: CDNs (3/5)

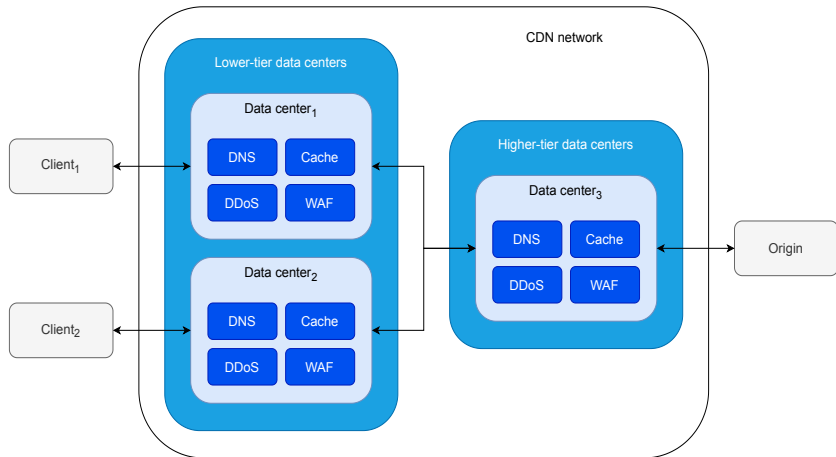


Figure 27: A CDN Reference Architecture (adopted from Cloudflare 2025)

Load Balancing Revisited: CDNs (4/5)

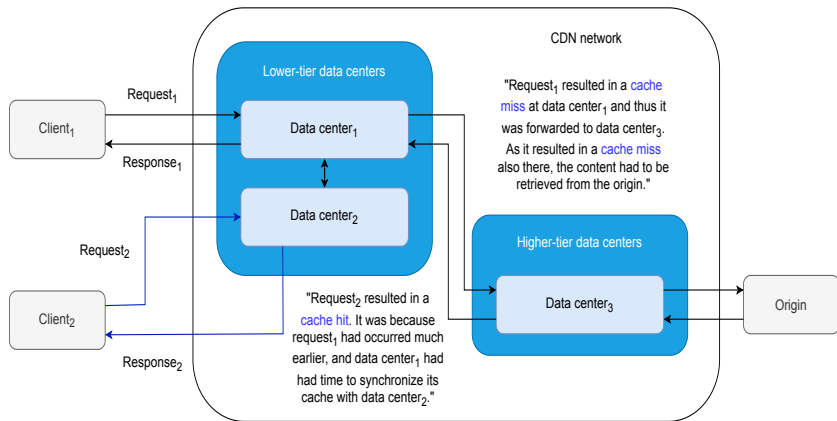


Figure 28: Caching in the CDN Ref. Architecture (from Cloudflare 2025)

Load Balancing Revisited: CDNs (5/5)

- ▶ Though, CDNs may conflict with other QAs, such as usability for clients, and the situation has recently become worse due to the AI crawlers noted in the previous lecture

portal.findresearcher.sdu.dk

❗ Please unblock challenges.cloudflare.com to proceed.

Ray ID: 9c77a33219d08d51

Performance & security by [Cloudflare](#)

Figure 29: Visiting Our Home University via Elsevier via Cloudflare

Load Balancing Revisited: SDN (1/6)

- ▶ It should be understood that most of what has been said is configured and managed via software; hence, a concept of **software-defined networking** (SDN)
- ▶ For the course's purposes, SDN is important because it bridges the low-level machine view to software-defined middleware and from there to large-scale networking view
- ▶ To illustrate SDNs, a few *simple* examples are presented with the open source OpenStack project intended for building infrastructure-as-a-service (IaaS) solutions

Load Balancing Revisited: SDN (2/6)

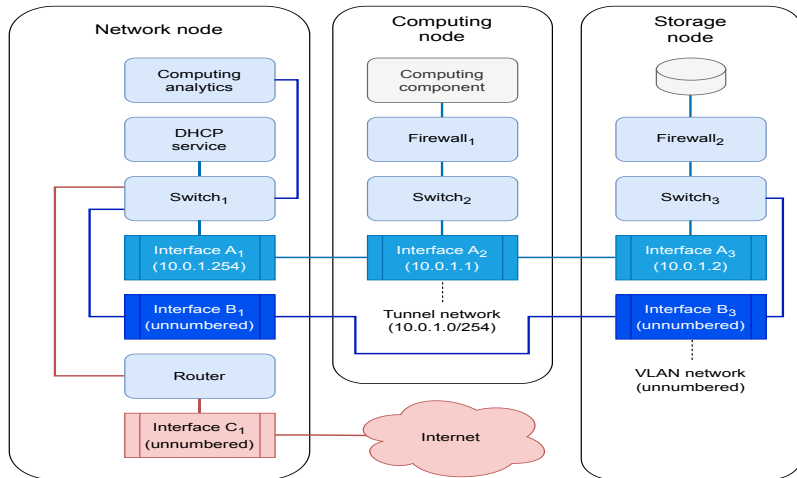


Figure 30: An Example SDN Network (adopted from OpenStack 2016)

Question

What Might Be a Design Justification
for the Previous SDN-Based System?

Load Balancing Revisited: SDN (3/6)

- ▶ While there are no right or wrong answers, a possible answer could be that the computing node writes its results to the storage node only via the tunnel network, whereas all read operations from the Internet are routed via the VLAN network
 - The network node's interface A_1 is required for giving DHCP leases to the computing and storage nodes
- ▶ Thus, **security** could be argued to improve as the computing node is isolated from read operations, but it could also be that **latency** varies between the tunnel and VLAN networks
- ▶ Thus, all in all, the previous example demonstrates how **network segmentation** aligns with the *isolation* and *encapsulation* tactics that was discussed in the third lecture

Load Balancing Revisited: SDN (4/6)

- ▶ OpenStack (2023) can also be used for simple load-balancing at the topmost, seventh (application) layer in the OSI model
- ▶ There are three policy actions for HTTP:
 1. REJECT: request is denied
 2. REDIRECT_TO_URL: request is redirected to a given URL
 3. REDIRECT_TO_POOL: request is redirected to a given pool
- ▶ Above, the term “pool” is a shorthand for one or more back-end servers, including web servers in particular
- ▶ Among the rule types are HEADER (matches fields in HTTP request headers), PATH (matches HTTP request URIs), and FILE_TYPE (matches specific file types, such as .png)

Load Balancing Revisited: SDN (5/6)

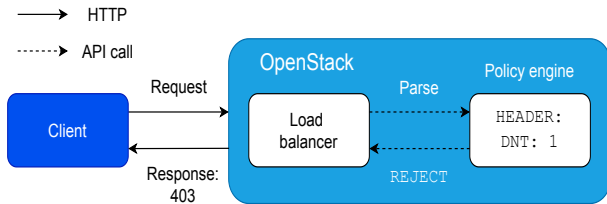


Figure 31: Rejecting Privacy-Aware Users with OpenStack

Load Balancing Revisited: SDN (6/6)

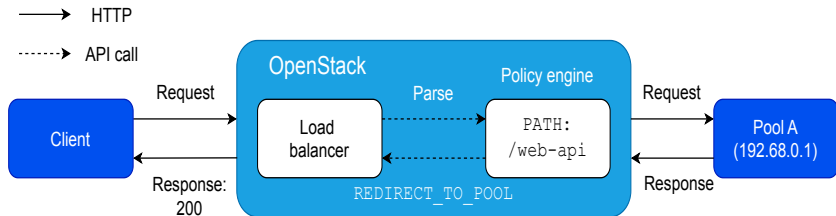
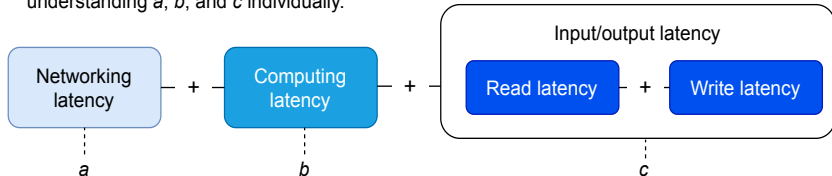


Figure 32: Redirecting to a Pool with OpenStack

Latency and Throughput Revisited (1/5)

Although the overall latency can be seen to be $y = a + b + c$ (in some time unit), scalability and performance QAs require understanding a , b , and c individually.

With Kubernetes, OpenStack, and related solutions, one should further understand that a , b , and c will vary according to the proportions of real hardware and virtualization used.



However, latency depends also on throughput: for instance, if the network has extreme throughput and minimal latency, it may be likely that the choke points will be seen via b and c .

Figure 33: Further Takeaways from the Earlier Examples

Latency and Throughput Revisited (2/5)

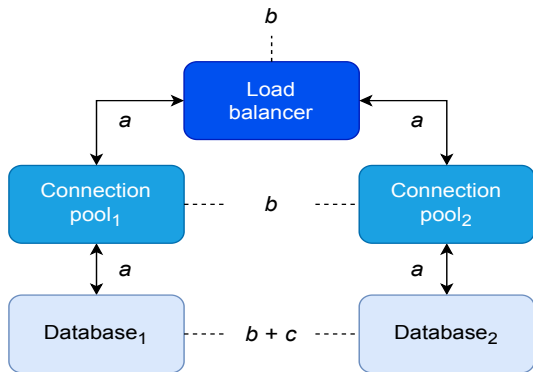


Figure 34: Hence, Note That a , b , c Can All Be Further Decomposed

Latency and Throughput Revisited (3/5)

- ▶ Analogously to the previous lecture on performance, at a high-level of abstraction, we can think about “**time scalability**” (or load scalability) and “**space scalability**”
- ▶ However, these are related: with **big data**, the sizes of objects are large to transmit via a network (increases a), difficult to compute due to memory constraints (increases b), and slow to read and write (increases c)
- ▶ With this terminology, we can “*regard a system as having space-time scalability if it continues to function gracefully as the number of objects it encompasses increases by **orders of magnitude***” (Bondi 2000, p. 3; boldfacing added)

Latency and Throughput Revisited (4/5)

- ▶ The earlier point about geofencing bring the following two additional scalability dimensions noted by Bondi (2000, p. 3):
 1. **Distance scalability**: a system “is distance scalable if it works well over long distances as well as short distances”
 2. **Speed/distance scalability**: a system “is speed/distance scalable if it works well over long distances as well as short distances at high and low speeds”
- ▶ For instance, a system using Bluetooth has neither

Latency and Throughput Revisited (5/5)

- ▶ We have already emphasized scalability from a business perspective, but there is a further theoretical perspective:

*“We think of a system as being **structurally scalable** if its implementation or standards do not impede the growth of the number of objects it encompasses, or at least will not do so within a chosen time frame. This is a relative term, because scalability depends on the number of objects of interest now relative to the number of objects later.”* (Bodi 2000, p. 3; boldfacing added)

- ▶ One way to interpret the quote is a reversal of what was said in the first lecture: that is, an architecture may be designed scalable but an implementation hinders reaching that potential

Performance and Scalability Diagnosis (1/4)

- ▶ Bodi (2000) also gives some practical diagnosis guidelines:
 1. Identify and diagnose all cases in which components—or even a whole system—could enter into some **absorbing state** (such as a deadlock) or a **wholesale failure**
 2. Identify a system's **parts that can run in parallel**
 3. Identify **self-expanding performance bottlenecks** (cf. cascading failures but from a performance viewpoint)
 4. Identify and eliminate **unproductive execution cycles**
 5. Seek to minimize **temporary execution cycles**
 6. Identify whether scalability problems are due to **scheduling**

Performance and Scalability Diagnosis (2/4)

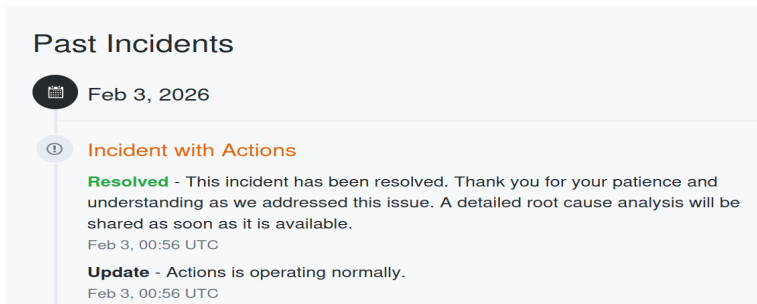
- ▶ As sensible as the previous guidelines are, they are challenging during development as they mostly require a full **runtime deployment**, and (continuous) *deployability in itself is often a challenge* for large-scale systems such as Kubernetes clusters
- ▶ They may also require a deployment on **real hardware**
 - For instance, at some point there were plans to implement *automated benchmarks* via the NetBSD's large-scale testing framework, but these were dropped as robust benchmarks could not be done in virtualization environments
- ▶ Diagnosing performance and scalability is also quite different from traditional debugging, perhaps being closer to something called “**statistical debugging**” (Somg & Shan 2014)

Performance and Scalability Diagnosis (3/4)

- ▶ Bansal et al. (2020) also note **logs** as a problem: they are often unstructured or semi-structured; come from various sources sometimes with high velocity; are voluminous; have high dimensionality; may not be retained long enough; etc.
- ▶ In addition, **performance regressions** are common, but, when compared to “normal bugs”, they are difficult to detect even with sophisticated DevOps practices

Performance and Scalability Diagnosis (4/4)

- Therefore, **retrospective reasoning and triaging** are often required; *what is the impact; how many users are affected; is the problem local or global; is it a known bug; has it happened before; and so forth and so on?*



The screenshot shows the 'Past Incidents' section of the GitHub Status page. It features a header 'Past Incidents' with a calendar icon and the date 'Feb 3, 2026'. Below this, there is a section titled 'Incident with Actions' with an information icon. The incident is marked as 'Resolved' in green text, followed by a message: 'This incident has been resolved. Thank you for your patience and understanding as we addressed this issue. A detailed root cause analysis will be shared as soon as it is available.' The timestamp 'Feb 3, 00:56 UTC' is shown. Below this, an 'Update' section states 'Actions is operating normally.' with the same timestamp 'Feb 3, 00:56 UTC'.

Past Incidents

 Feb 3, 2026

 **Incident with Actions**

Resolved - This incident has been resolved. Thank you for your patience and understanding as we addressed this issue. A detailed root cause analysis will be shared as soon as it is available.
Feb 3, 00:56 UTC

Update - Actions is operating normally.
Feb 3, 00:56 UTC

Figure 35: <https://www.githubstatus.com/?todayis=2026-02-02>

Questions?

Case #7

- ▶ Design an alert system with Prometheus for providing *instant notifications* to a system administrator in case *two or more* Kubernetes *nodes* or pods become dead while in operation
 - Illustrate the design with **two architectural views**: one addressing the design at the side of Kubernetes and the other at the Prometheus' side
 - The design does not have to be 100% accurate technically; rather, consider that it would be a **blueprint** that you would provide as a software architect to an implementation team
 - Rationalize also other QAs, such as complexity

Additional References (1/3)

1. Amdahl, G. (1967): Validity of the Single Processor Approach to Achieving Large Scale Computing Capabilities. AFIPS,
<https://dl.acm.org/doi/epdf/10.1145/1465482.1465560>
2. Gustafson, J. L. (1988): Reevaluating Amdahl's Law. Communications of the ACM, Volume 31(5), pp. 532–533.
<https://dl.acm.org/doi/epdf/10.1145/42411.42415>
3. Hill, M. D. & Marty, M. R. (2008): Amdahl's Law in the Multicore Era. Computer 41(7), pp. 33–38,
<https://minds.wisconsin.edu/bitstream/handle/1793/60554/TR1593.pdf>
4. Microsoft (2025): Cloud Design Patterns.
<https://learn.microsoft.com/en-us/azure/architecture/patterns/>
5. Maia, T. & Correia, F. F. (2022): Service Mesh Patterns. EuroPLop, pp. 1–12,
<https://dl.acm.org/doi/pdf/10.1145/3551902.3551962>
6. Microsoft (2020): Sidecar Pattern in Action.
<https://techcommunity.microsoft.com/blog/azurearchitectureblog/sidecar-pattern-in-action/1714983>

Additional References (2/3)

1. Google (2024): Best Practices for Running Cost-Optimized Kubernetes Applications on GKE. <https://docs.cloud.google.com/architecture/best-practices-for-running-cost-effective-kubernetes-applications-on-gke/>
2. Kubernetes (2024a): Horizontal Pod Autoscaling. <https://kubernetes.io/docs/concepts/workloads/autoscaling/horizontal-pod-autoscale/>
3. Dinesh, S. (2018): Kubernetes Best Practices: Resource Requests and Limits. Google Cloud Blog, <https://cloud.google.com/blog/products/containers-kubernetes/kubernetes-best-practices-resource-requests-and-limits>
4. Bufalino, J. & Martin-Navarro, J. L. & Di Francesco, M. & Aura, T. (2025): Inside Job: Defending Kubernetes Clusters Against Network Misconfigurations. Proceedings of the ACM on Networking 3(CoNEXT3), pp. 1–25, <https://dl.acm.org/doi/epdf/10.1145/3749220>
5. Kubernetes (2024b): Considerations for Large Clusters. <https://kubernetes.io/docs/setup/best-practices/cluster-large/>
6. Ruohonen, J. (2020): Measuring Basic Load-Balancing and Fail-Over Setups for Email Delivery via DNS MX Records. IFIP Networking, pp. 815–820, <https://arxiv.org/pdf/2002.10731>

Additional References (3/3)

1. Cloudflare (2025): Content Delivery Network (CDN) Reference Architecture.
<https://developers.cloudflare.com/reference-architecture/architectures/cdn/>
2. OpenStack (2016): Scenario: Legacy with Open vSwitch. https://docs.openstack.org/kilo/networking-guide/scenario_legacy_ovs.html
3. OpenStack (2023): Layer 7 Load Balancing.
<https://docs.openstack.org/octavia/latest/user/guides/l7.html>
4. Bondi, A. B. (2000): Characteristics of Scalability and Their Impact on Performance. WOSP, pp. 1–10,
<https://dl.acm.org/doi/pdf/10.1145/350391.350432>
5. Song, L. & Lu, Shan (2014): Statistical Debugging for Real-World Performance Problems. OOPSLA, pp. 561–578,
<https://dl.acm.org/doi/pdf/10.1145/2714064.2660234>
6. Bansal, C. & Renganathan, S. & Asudani, A. & Midy, O. & Janakiraman, M. (2020): DeCaf: Diagnosing and Triaging Performance Issues in Large-Scale Cloud Services. ICSE-SEIP, pp. 201–210,
<https://dl.acm.org/doi/pdf/10.1145/3377813.3381353>